Date:	
Program 8	Implement sort algorithms (Quick sort, Insertion sort)

```
#include <stdio.h>
#include <stdlib.h>

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
void swap(int* a, int* b) {
    int t = *a;
    *a = *b;
    *b = t;
}
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    int j;
    for (j = low; j <= high - 1; j++) {
```

```
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
  }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
void printArray(int arr[], int n) {
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
int main() {
    int n, i, choice;
    int arr[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("\n--- Sorting Menu ---\n");
```

```
    printf("1. Insertion Sort\n");
    printf("2. Quick Sort\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            insertionSort(arr, n);
            printf("Sorted array using Insertion Sort: \n");
            break;
        case 2:
            quickSort(arr, 0, n - 1);
            printf("Sorted array using Quick Sort: \n");
            break;




        default:
            printf("Invalid choice!\n");
            return 0;
    }
    printArray(arr, n);
    return 0;
}
```

```
"D:\DFS Lab Programs\Inserti

Enter number of elements: 5
Enter 5 elements:
85 6 36 79 42

--- Sorting Menu ---
1. Insertion Sort
2. Quick Sort
Enter choice: 1
Sorted array using Insertion Sort:
6 36 42 79 85
Press any key to continue . . .
```

```
"D:\DFS Lab Programs\Inserti

Enter number of elements: 5
Enter 5 elements:
100 41 28 6 81

--- Sorting Menu ---
1. Insertion Sort
2. Quick Sort
Enter choice: 2
Sorted array using Quick Sort:
6 28 41 81 100
Press any key to continue . . .
```

Date:	
Program 9	Program to Implement Linear Search & Binary Search

```
#include <stdio.h>

#include <stdlib.h>


int linearSearch(int arr[], int n, int key) {

    int i;

    for (i = 0; i < n; i++) {

        if (arr[i] == key) {

            return i;

        }

    }

    return -1;

}


int binarySearch(int arr[], int n, int key) {

    int low = 0, high = n - 1;

    while (low <= high) {

        int mid = low + (high - low) / 2;


        if (arr[mid] == key)
```

```
            return mid;   // ? fixed here


        if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}


int main() {
    int n, i, key, choice, result;
    int arr[100];


    printf("Enter number of elements: ");
    scanf("%d", &n);


    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }


    printf("\n--- Search Menu ---\n");
```

```
printf("1. Linear Search\n");
printf("2. Binary Search (Ensure array is sorted!)\n");
printf("Enter choice: ");
scanf("%d", &choice);

printf("Enter element to search: ");
scanf("%d", &key);

switch (choice) {
    case 1:
        result = linearSearch(arr, n, key);
        break;
    case 2:
        result = binarySearch(arr, n, key);
        break;
    default:
        printf("Invalid choice!\n");
        return 0;
}
```

```
    if (result != -1)
        printf("Element found at index: %d (Position: %d)\n", result, result + 1);
    else
        printf("Element not found in the array.\n");


    return 0;
}
```

```
"D:\DFS Lab Programs\Linear_

Enter number of elements: 5
Enter 5 elements:
65 46 2 97 350

--- Search Menu ---
1. Linear Search
2. Binary Search (Ensure array is sorted!)
Enter choice: 1
Enter element to search: 2
Element found at index: 2 (Position: 3)
Press any key to continue . . .
```

```
"D:\DFS Lab Programs\Linear_

Enter number of elements: 6
Enter 6 elements:
25 6 78 650 35 10

--- Search Menu ---
1. Linear Search
2. Binary Search (Ensure array is sorted!)
Enter choice: 1
Enter element to search: 650
Element found at index: 3 (Position: 4)
Press any key to continue . . .
```

```
"D:\DFS Lab Programs\Linear_

Enter number of elements: 5
Enter 5 elements:
1 46 58 77 100

--- Search Menu ---
1. Linear Search
2. Binary Search (Ensure array is sorted!)
Enter choice: 2
Enter element to search: 77
Element found at index: 3 (Position: 4)
Press any key to continue . . .
```

```
"D:\DFS Lab Programs\Linear_

Enter number of elements: 4
Enter 4 elements:
108 119 165 250

--- Search Menu ---
1. Linear Search
2. Binary Search (Ensure array is sorted!)
Enter choice: 2
Enter element to search: 108
Element found at index: 0 (Position: 1)
Press any key to continue . . .
```

Date:	
Program 10	Program to demonstrate operations on Binary Trees: (Create, Insert, Delete, Process)

```
#include  <stdio.h>

#include <stdlib.h>

struct Node {

    int data;

    struct Node *left, *right;

};

struct Node* createNode(int data) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = data;

    newNode->left = newNode->right = NULL;

    return newNode;

}

void inorder(struct Node* temp) {

    if (temp == NULL) return;

    inorder(temp->left);

    printf("%d ", temp->data);

    inorder(temp->right);

}

struct Node* insert(struct Node* root, int data) {
```

```
    if (root == NULL) {
        return createNode(data);
    }
    struct Node* queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;
    while (front < rear) {
        struct Node* temp = queue[front++];
 if (temp->left == NULL) {
            temp->left = createNode(data);
            break;
        } else {
            queue[rear++] = temp->left;
        }
        if (temp->right == NULL) {
            temp->right = createNode(data);
            break;
        } else {
            queue[rear++] = temp->right;
        }
    }
    return root;
}
```

```
void deleteDeepest(struct Node* root, struct Node* d_node) {
    struct Node* queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;
    while (front < rear) {
        struct Node* temp = queue[front++];
        if (temp == d_node) {
            temp = NULL;
            free(d_node);
            return;
        }
        if (temp->right) {
            if (temp->right == d_node) {
                temp->right = NULL;
                free(d_node);
                return;
            } else queue[rear++] = temp->right;
      }
        if (temp->left) {
            if (temp->left == d_node) {
                temp->left = NULL;
                free(d_node);
                return;
```

```
            } else queue[rear++] = temp->left;
        }
    }
}
struct Node* deletion(struct Node* root, int key) {
    if (root == NULL) return NULL;
    if (root->left == NULL && root->right == NULL) {
        if (root->data == key) return NULL;
        else return root;
    }
    struct Node* queue[100];
    int front = 0, rear = 0;
    queue[rear++] = root;
    struct Node* temp;
    struct Node* key_node = NULL;
    while (front < rear) {
        temp = queue[front++];
        if (temp->data == key) key_node = temp;
        if (temp->left) queue[rear++] = temp->left;
        if (temp->right) queue[rear++] = temp->right;
    } if (key_node != NULL) {
        int x = temp->data;
        deleteDeepest(root, temp);
```

```
        key_node->data = x;
    } return root;
}
int main() {
    struct Node* root = NULL;
    int choice, val,n,i;
    while (1) {
        printf("\n1. Insert\n2. Delete\n3. Display (Inorder)\n4. Exit\n");
        printf("Choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
             printf("enter no of elements to insert:\t");
             scanf("%d",&n);
             for(i=0;i<n;i++){
              printf("Value to insert:");
                 scanf("%d", &val);
                 root = insert(root, val);
             }
                break;
            case 2:
                printf("Value to delete: ");
                scanf("%d", &val);
```

```
            root = deletion(root, val);
            break;
        case 3:
            printf("Tree: ");
            inorder(root);
            printf("\n");
            break;
        case 4:
            exit(0);
    }
}
  return 0;
}
```

```
1. Insert
2. Delete
3. Display (Inorder)
4. Exit
Choice: 3
Tree: 10 98 64 51

1. Insert
2. Delete
3. Display (Inorder)
4. Exit
Choice: 4
Press any key to continue . . .
```

```
1. Insert
2. Delete
3. Display (Inorder)
4. Exit
Choice: 3
Tree: 10 98 33 64 51

1. Insert
2. Delete
3. Display (Inorder)
4. Exit
Choice: 2
Value to delete: 33
```

```
"D:\DFS Lab Programs\Binary

1. Insert
2. Delete
3. Display (Inorder)
4. Exit
Choice: 1
enter no of elements to insert: 5
Value to insert:64
Value to insert:98
Value to insert:51
Value to insert:10
Value to insert:33
```

Date:	
Program 11	Program to implement Binary Search Tree operations: Insert, Traversals (Inorder, Preorder, Postorder)

```
#include  <stdio.h>

#include <stdlib.h>

struct Node {

    int data;

    struct Node *left, *right;

};

struct Node* createNode(int value) {

    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;

    newNode->left = newNode->right = NULL;

    return newNode;

}

struct Node* insert(struct Node* root, int value) {

    if (root == NULL) {

        return createNode(value);

    }

    if (value < root->data) {

        root->left = insert(root->left, value);

    } else if (value > root->data) {
```

```
        root->right = insert(root->right, value);
    }
    return root;
}
void inorder(struct Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
            }
}
void preorder(struct Node* root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
void postorder(struct Node* root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
```

```
    }
}
int main() {
    struct Node* root = NULL;
    int choice, value;

    printf("--- Binary Search Tree Operations ---\n");
    while(1) {
        printf("\n1. Insert\n2. Inorder\n3. Preorder\n4. Postorder\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                root = insert(root, value);
                break;
                  case 2:
                printf("Inorder Traversal: ");
                inorder(root);
                printf("\n");
                break;
```

```
        case 3:
            printf("Preorder Traversal: ");
            preorder(root);
            printf("\n");
            break;
        case 4:
            printf("Postorder Traversal: ");
            postorder(root);
            printf("\n");
            break;
        case 5:
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
  }
  return 0;
}
```

```
"D:\DFS Lab Programs\BST_O
--- Binary Search Tree Operations

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 1
Enter value to insert: 3

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 1
Enter value to insert: 12

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 1
Enter value to insert: 46
```

```
1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 1
Enter value to insert: 67

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 1
Enter value to insert: 90
```

```
1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 2
Inorder Traversal: 3 12 46 67 90

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 3
Preorder Traversal: 3 12 46 67 90

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 4
Postorder Traversal: 90 67 46 12 3

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Exit
Enter choice: 5
Press any key to continue . . .
```

Date:	
Program 12	Program to display the Adjacency Matrix of a Graph

```
#include <stdio.h>

#define MAX 100


int main()

{

    int adj[MAX][MAX];

    int vertices, edges;

    int src, dest,i,j;

    printf("Enter the number of vertices (max 100): ");

    scanf("%d", &vertices);

    printf("Enter the number of edges: ");

    scanf("%d", &edges);

    for (i = 0; i < vertices; i++) {

        for (j = 0; j < vertices; j++) {

            adj[i][j] = 0;

        }

    }

    printf("\nEnter edges (source destination):\n");
```

```
    for (i = 0; i < edges; i++) {
        printf("Edge %d: ", i + 1);
        scanf("%d %d", &src, &dest);
        adj[src][dest] = 1;
    }
    printf("Adjacency Matrix:\n");
    printf("     ");
    for (i = 0; i < vertices; i++) {
      printf(" v%d ", i);
    }
    printf("\n");
    for (i = 0; i < vertices; i++)
    {
        printf("v%d  ", i);
        for (j = 0; j < vertices; j++)
       {
            printf(" %d  ", adj[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

```
"D:\DFS Lab Programs\G_Adj
Enter the number of vertices (max 100): 6
Enter the number of edges: 7

Enter edges (source destination):
Edge 1: 2
3
Edge 2: 4
2
Edge 3: 5
6
Edge 4: 1
3
Edge 5: 6
4
Edge 6: 7
8
Edge 7: 7
5
```

```
Adjacency Matrix:
     v0  v1  v2  v3  v4  v5
v0   0   0   0   0   0   0
v1   0   0   0   1   0   0
v2   0   0   0   1   0   0
v3   0   0   0   0   0   0
v4   0   0   1   0   0   0
v5   0   0   0   0   0   0
Press any key to continue . . .
```

Date:	
Program 13 (a)	Program to perform Depth First Traversal (DFS) on a Graph

```
#include <stdio.h>

#define MAX 100


int adj[MAX][MAX];

int visited[MAX];

int vertices;

void DFS(int vertex) {

 int i;

    printf("%d ", vertex);

    visited[vertex] = 1;

    for (i = 0; i < vertices; i++) {

        if (adj[vertex][i] == 1 && !visited[i]) {

            DFS(i);

   } }}

void displayMatrix() {

 int i,j;

    printf("Adjacency Matrix:\n");

    printf("      ");
```

```
    for (i = 0; i < vertices; i++) {
        printf(" v%d ", i);
    }

    printf("\n");
    for (i = 0; i < vertices; i++) {
        printf("v%d  ", i);
        for (j = 0; j < vertices; j++) {
            printf(" %d  ", adj[i][j]);
        }
          printf("\n");
    }
}
int main() {
    int edges, src, dest, start,i,j;
    printf("Enter the number of vertices (max 100): ");
    scanf("%d", &vertices);
    printf("Enter the number of edges: ");
    scanf("%d", &edges);
    for (i = 0; i < vertices; i++) {
        for (j = 0; j < vertices; j++) {
            adj[i][j] = 0;
        }
```

```
        visited[i] = 0;
    }
    printf("\nEnter edges (source destination):\n");
    for (i = 0; i < edges; i++) {
        printf("Edge %d: ", i + 1);
        scanf("%d %d", &src, &dest);
        adj[src][dest] = 1;
    }
    displayMatrix();
    printf("\nEnter starting vertex for DFS: ");
    scanf("%d", &start);
    for (i = 0; i < vertices; i++) {
        visited[i] = 0;
    }
    printf("\nDFS Traversal starting from vertex %d: ", start);
    DFS(start);
    printf("\n");
    return 0;
}
```

```
"D:\DFS Lab Programs\Depth_

Enter the number of vertices (max 100): 7
Enter the number of edges: 8

Enter edges (source destination):
Edge 1: 1
3
Edge 2: 1
6
Edge 3: 4
7
Edge 4: 4
2
Edge 5: 7
5
Edge 6: 2
5
Edge 7: 6
3
Edge 8: 5
4
```

```
Adjacency Matrix:
      v0  v1  v2  v3  v4  v5  v6
v0    0   0   0   0   0   0   0
v1    0   0   0   1   0   0   1
v2    0   0   0   0   0   1   0
v3    0   0   0   0   0   0   0
v4    0   0   1   0   0   0   0
v5    0   0   0   0   1   0   0
v6    0   0   0   1   0   0   0

Enter starting vertex for DFS: 1

DFS Traversal starting from vertex 1: 1 3 6
Press any key to continue . . .
```

Date:	
Program 13 (b)	Program to perform Breadth First Traversal (BFS) on a Graph

```
#include <stdio.h>

#define MAX 100


int adj[MAX][MAX];

int visited[MAX];

int vertices;


int queue[MAX];

int front = -1, rear = -1;


void enqueue(int vertex) {

    if (rear == MAX - 1) {

        printf("Queue Overflow\n");

        return;

    }

    if (front == -1)

        front = 0;

    rear++;
```

```
    queue[rear] = vertex;
}

int dequeue() {
    if (front == -1 || front > rear) {
        printf("Queue Underflow\n");
        return -1;
    }
    int vertex = queue[front];
    front++;
    return vertex;
}

int isEmpty() {
    return (front == -1 || front > rear);
}

void BFS(int start) {
    int i;

    for (i = 0; i < vertices; i++) {
        visited[i] = 0;
    }
```

```
    front = rear = -1;

    printf("%d ", start);
    visited[start] = 1;
    enqueue(start);

    while (!isEmpty()) {
        int current = dequeue();

        for (i = 0; i < vertices; i++) {
            if (adj[current][i] == 1 && !visited[i]) {
                printf("%d ", i);
                visited[i] = 1;
                enqueue(i);
            }
        }
    }
}

void displayMatrix() {
    int i, j;
```

```
    printf("Adjacency Matrix:\n");
    printf("      ");
    for (i = 0; i < vertices; i++) {
        printf(" v%d ", i);
    }
    printf("\n");

    for (i = 0; i < vertices; i++) {
        printf("v%d   ", i);
        for (j = 0; j < vertices; j++) {
            printf(" %d  ", adj[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int edges, src, dest, start;
    int i, j;

    printf("Enter the number of vertices (max 100): ");
    scanf("%d", &vertices);
```

```
    printf("Enter the number of edges: ");
    scanf("%d", &edges);

    for (i = 0; i < vertices; i++) {
        for (j = 0; j < vertices; j++) {
            adj[i][j] = 0;
        }
        visited[i] = 0;
    }

    printf("\nEnter edges (source destination):\n");
    for (i = 0; i < edges; i++) {
        printf("Edge %d: ", i + 1);
        scanf("%d %d", &src, &dest);
        adj[src][dest] = 1;
    }

    displayMatrix();

    printf("\nEnter starting vertex for BFS: ");
    scanf("%d", &start);

    printf("\nBFS Traversal starting from vertex %d: ", start);
```

```
    BFS(start);

    printf("\n");


    return 0;

}
```

```
"D:\DFS Lab Programs\BFS.ex

Enter the number of vertices (max 100): 7
Enter the number of edges: 6

Enter edges (source destination):
Edge 1: 1
3
Edge 2: 4
2
Edge 3: 5
6
Edge 4: 3
4
Edge 5: 5
2
Edge 6: 4
3
```

```
Adjacency Matrix:
      v0  v1  v2  v3  v4  v5  v6
v0    0   0   0   0   0   0   0
v1    0   0   0   1   0   0   0
v2    0   0   0   0   0   0   0
v3    0   0   0   0   1   0   0
v4    0   0   1   1   0   0   0
v5    0   0   1   0   0   0   1
v6    0   0   0   0   0   0   0

Enter starting vertex for BFS: 1

BFS Traversal starting from vertex 1: 1 3 4 2
Press any key to continue . . .
```

Date:	
Program 14	Program to implement B tree and perform operations (Insert,Search,Display)

```
#include <stdio.h>
#include <stdlib.h>

#define T 2

typedef struct BTree
{
   int n;
   int leaf;
   int key[2 * T - 1];
   struct BTree *child[2 * T];
} node;

node *root = NULL;

node* createNode(int leaf)
{
 int i;
   node *newNode = (node*)malloc(sizeof(node));
   newNode->leaf = leaf;
   newNode->n = 0;
   for (i = 0; i < 2 * T; i++)
      newNode->child[i] = NULL;
   return newNode;
}
void traverse(node *ptr)
```

```
{
  if (ptr != NULL)
    {
        int i;
        for (i = 0; i < ptr->n; i++)
        {
            if (!ptr->leaf)
                traverse(ptr->child[i]);
            printf("%d ", ptr->key[i]);
        }
        if (!ptr->leaf)
            traverse(ptr->child[i]);
    }
}
node* search(node *ptr, int k)
{
    int i = 0;
    while (i < ptr->n && k > ptr->key[i])
        i++;
    if (i < ptr->n && k == ptr->key[i])
        return ptr;
    if (ptr->leaf)
        return NULL;
    return search(ptr->child[i], k);
}
void splitChild(node *parent, int i, node *fullChild)
{
 int j;
    node *newChild = createNode(fullChild->leaf);
    newChild->n = T - 1;
    for (j = 0; j < T - 1; j++)
```

```
        newChild->key[j] = fullChild->key[j + T];
    if (!fullChild->leaf)
        {
        for (j = 0; j < T; j++)
            newChild->child[j] = fullChild->child[j + T];
    }
    fullChild->n = T - 1;
    for (j = parent->n; j >= i + 1; j--)
        parent->child[j + 1] = parent->child[j];
    parent->child[i + 1] = newChild;
    for (j = parent->n - 1; j >= i; j--)
        parent->key[j + 1] = parent->key[j];
    parent->key[i] = fullChild->key[T - 1];
    parent->n = parent->n + 1;
}
void insertNonFull(node *ptr, int k)
{
    int i = ptr->n - 1;
    if (ptr->leaf)
    {
        while (i >= 0 && k < ptr->key[i])
        {
            ptr->key[i + 1] = ptr->key[i];
            i--;
        }
        ptr->key[i + 1] = k;
        ptr->n = ptr->n + 1;
    }
    else
    {
        while (i >= 0 && k < ptr->key[i])
```

```
            i--;
        i++;
        if (ptr->child[i]->n == 2 * T - 1)
 {
            splitChild(ptr, i, ptr->child[i]);

            if (k > ptr->key[i])
                i++;
        }
        insertNonFull(ptr->child[i], k);
    }
}
void insert(int k)
{
    if (root == NULL)
    {
        root = createNode(1);
        root->key[0] = k;
        root->n = 1;
        return;
    }
    if (search(root, k) != NULL)
    {
        printf("Duplicate key not allowed.\n");
        return;
    }
    if (root->n == 2 * T - 1)
    {
        node *newRoot = createNode(0);
        newRoot->child[0] = root;
        splitChild(newRoot, 0, root);
```

```
        int i = 0;
        if (k > newRoot->key[0])
            i++;
        insertNonFull(newRoot->child[i], k);
         root = newRoot;
    }
    else
    {
        insertNonFull(root, k);
    }
}
int main()
{
    int ch, val;
    node *found;

    while (1)
    {
        printf("\n--- B-Tree Menu ---\n");
        printf("1. Insert\n");
        printf("2. Search\n");
        printf("3. Traverse\n");
        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &ch);
        switch (ch)
        {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &val);
                insert(val);
```

```
        break;
    case 2:
        if (root == NULL)
        {
            printf("Tree is empty.\n");
    }
        else
        {
            printf("Enter value to search: ");
            scanf("%d", &val);
            found = search(root, val);

            if (found != NULL)
                printf("%d found in B-Tree.\n", val);
            else
                printf("%d not found in B-Tree.\n", val);
        }
        break;
    case 3:
        if (root == NULL)
            printf("Tree is empty.\n");
        else
        {
            printf("Traversal: ");
            traverse(root);
            printf("\n");
        }
        break;
    case 4:
        printf("Exiting...\n");
        exit(0);
```

```
        default:
            printf("Invalid choice.\n");
    }
  }
  return 0;
}
```

```
"D:\DFS Lab Programs\BTree_

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 64

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 25

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 99
```

```
--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 45

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 70

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 1
Enter value to insert: 52
```

```
--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 3
Traversal: 25 45 52 64 70 99

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 2
Enter value to search: 52
52 found in B-Tree.
```

```
--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 2
Enter value to search: 100
100 not found in B-Tree.

--- B-Tree Menu ---
1. Insert
2. Search
3. Traverse
4. Exit
Enter choice: 4
Exiting...
Press any key to continue . . .
```

Date:	
Program 15	Program to perform Hashing using Mid-Square Method and Linear Probing (Collision Resolution)

```
#include  <stdio.h>

#include <stdlib.h>

#include  <math.h>

#define SIZE 10

#define EMPTY -1


int hashTable[SIZE];

int midSquareHash(int key) {

    long long square = (long long)key * key;

    int mid;

    if (square < 10) {

        mid = (int)square;

    } else {

        mid = (int)(square / 10) % 10;

    }

    return mid % SIZE;

}

void insert(int key) {

    int index = midSquareHash(key);
```

```
    int i;
    for (i = 0; i < SIZE; i++) {
        int probeIndex = (index + i) % SIZE;


        if (hashTable[probeIndex] == EMPTY) {
            hashTable[probeIndex] = key;
            printf("Key %d inserted at index %d\n", key, probeIndex);
                return;
        }
    }
    printf("Hash Table is full! Cannot insert %d\n", key);
}
void display() {
    int i;
    printf("\nHash Table Contents:\n");
    printf("Index\tKey\n");
    for (i = 0; i < SIZE; i++) {
        if (hashTable[i] != EMPTY)
            printf("%d\t%d\n", i, hashTable[i]);
        else
            printf("%d\t---\n", i);
    }
}
```

```
int main() {
    int i, choice, key;
    for (i = 0; i < SIZE; i++) {
        hashTable[i] = EMPTY;
    }
    while (1) {
        printf("\n1. Insert Key\n2. Display Table\n3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter key to insert: ");
                scanf("%d", &key);
                insert(key);
                    break;
            case 2:
                display();
                break;
            case 3:
                exit(0);
```

```
        default:
            printf("Invalid choice!\n");
    }
  }
  return 0;
}
```

```
"D:\DFS Lab Programs\Hashir

1. Insert Key
2. Display Table
3. Exit
Enter choice: 1
Enter key to insert: 15
Key 15 inserted at index 2

1. Insert Key
2. Display Table
3. Exit
Enter choice: 1
Enter key to insert: 65
Key 65 inserted at index 3
```

```
1. Insert Key
2. Display Table
3. Exit
Enter choice: 1
Enter key to insert: 30
Key 30 inserted at index 0

1. Insert Key
2. Display Table
3. Exit
Enter choice: 1
Enter key to insert: 78
Key 78 inserted at index 8
```

```
1. Insert Key
2. Display Table
3. Exit
Enter choice: 2

Hash Table Contents:
Index   Key
0       30
1       ---
2       15
3       65
4       ---
5       ---
6       ---
7       ---
8       78
9       ---

1. Insert Key
2. Display Table
3. Exit
Enter choice: 3
Press any key to continue . . .
```